

Problems in Agile Trenches

Mira Kajko-Mattsson

Department of Computer and Systems Sciences

Stockholm University

+46-8-16 16 70

mira@dsv.su.se

ABSTRACT

Some agile voices have been raised against extensive documentation. They advocate its sparse use and production. In this paper, we report from agile evolution and maintenance trenches on the lifecycle problems related to system and process documentation. Our report gives an indication of the problems that have been encountered within 18 agile organizations.

Categories and Subject Descriptors

D.2.7 [Distribution, Maintenance and Enhancement]: Corrections, Documentation, Enhancement

General Terms

Management, Documentation.

Keywords

Documentation, oral communication, product related problems, process related problems, communication owner, communication complexity.

1. INTRODUCTION

Some “agile” voices have been raised against extensive documentation. They advocate its sparse use and production. Their motto is “*working products over comprehensive documentation*”, implying that one should focus more on quick code production rather than its documentation. [2, 3]

According to the agilists, documentation should be brief, however precise enough to be useful within software production. It should be limited to the core parts of the system. In its extreme case, it should encompass only source code and a set of user stories [2]. The remaining system structure may be communicated informally, mainly orally. There may be no requirement specifications nor design documents, or if there are any, then for the purpose of supporting the coding activity.

The agilists do not claim that system documentation is not unimportant. It is just less important than working product versions. Documentation is secondary; however, it should cover

enough information for supporting the communication. The agilists claim that the quality of the system is not mainly achieved via documentation but via the iterative development, continuous integration, systematic testing and refactoring. Regarding the process documentation, no voices have been raised whatsoever, despite the fact that process documentation is just as important as system documentation.

Agile development may be effective from a short-term perspective. By reducing the amount of documentation, one increases productivity and decreases the production cost. From the long-term perspective, however, it may involve certain problems.

This paper reports on problems that have been encountered in agile evolution and maintenance trenches. It studies 18 software organizations practicing some form of an agile method. Using a list of well-known problems within traditional evolution and maintenance, it investigates whether these problems have been encountered in agile evolution and maintenance trenches.

The remainder of this paper is structured as follows. Section 2 describes the research method taken to make our investigation. Section 3 provides background to the documentation domain. Section 4 lists and describes problems that have been encountered in traditional evolution and maintenance. Section 5 presents the status and problems that have been encountered in 18 agile organizations. Finally, Sections 6 and 7 make concluding remarks and suggestions for future research.

2. RESEARCH METHOD

In this section, we present the research method taken during our study. In Section 2.1, we list and describe the study phases. In Section 2.2, we discuss the validity of our results.

2.1 Study Phases

The state of documentation has for many years been a favorite subject. Different types of studies have revealed that documentation has been one of the largest problems within the software industry [7, 10, 13, 19]. Many problems have been identified due to unsatisfactory documentation in traditional software environments. Hence, as a first step, we have compiled a list of these problems. Using this list, we created a questionnaire. We then used students on a master level to interview agile organizations. The interviews were mainly conducted on a tête-à-tête basis. In some cases, however, they were made over the phone. The interview answers were recorded and written down by

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee.

ESEM’08, October 9–10, 2008, Kaiserslautern, Germany.

Copyright 2008 ACM 978-1-59593-971-5/08/10...\$5.00.

Table 1. Our questionnaire

Introductory questions		Process Documentation Questions	
1.	What is the name of your company?	24.	Do you document the development, evolution and maintenance processes?
2.	What is the number of employees?	25.	Do you use a tool supporting the process with the help of which you may easily document the process steps?
3.	What type of products do you develop?	26.	Is this tool tailored to the process, that is, does it follow the organizational process?
4.	What is the size of your products (in LOC, FP (function points), OP (object points), or number of programs/ systems)?	27.	On what level do you document the process? (1) whole process level, (2) phase level, (3) activity level?
Please observe that the interviewee may have difficulties in answering this question.		28.	With your process documentation, do you achieve sufficient insight into the process? Are you able to monitor and control the process?
5.	What agile method do you use?	29.	Are you able to analyze the process and identify its drawbacks and suggest solutions for its improvement?
6.	What is your role within the organisation?	30.	Do you conduct root cause analysis in your organization?
7.	Before going over to the agile method, what method did you use? RUP, waterfall, other.	31.	How do you do it?
System Documentation Questions		32.	Are you able to make root cause analyses for identifying the underlying causes of the problems using the historical process documentation as documented in the tool?
8.	Do you document your systems?	33.	Is it possible for management to follow the progress of each individual software engineer? In what way?
9.	What documentation items describe the system? Could you please list them? * Do you document the requirements, architecture, high-level design, low-level design, code, test plans, test cases, user manuals? * How thorough is this documentation? * detailed * coarse-grained level	34.	Do you inform the customers about the progress? On what basis?
10.	Is the level of documenting systems at your organization satisfactory for promoting the system knowledge? Please explain and motivate.	35.	Does your process follow any process standard?
11.	Have you noticed whether the engineers (developers) within your organization start losing track and knowledge of the system structure?	36.	How do you evaluate that the process within your organization is compliant with this standard?
12.	If yes, in what way does it show?	37.	With time, a software product gets more complex. The original developers are not involved in evolving it any longer and probably the historical process documentation is not recorded as well. By historical process documentation, I mean records of what had been done when developing and evolving the system and records of all important decisions such as for instance, choice of a certain architectural feature. Has it become more difficult to evolve and maintain the system?
13.	What problems do you get when key software engineers quit your organization?	38.	One claims that agile methods are effective when developing small systems in small teams. But many times these systems and the organizations scale up (become larger). Have you had any problems with scaling up of the system and the organization?
14.	How do the new hires learn the system? 1. How soon are they allowed to code? 2. Any problems with respect to the training of the new hires?	39.	What are these problems?
15.	Have you noticed whether the support technicians (on <i>Support Line</i> 1 and 2) within your organization start losing track of the system structure when supporting their customers in their daily operation?	40.	In what way have you tried to remedy them?
16.	In what way does it show?	41.	In an agile environment, it may be difficult to integrate the developers with testers. Due to short release time, the testers do not have any opportunity to properly prepare the testing plans and to thoroughly test the software. Also, lack of system documentation means that they have to base their testing work on oral communication. Have you noticed any problems with respect to the testers and their ability to thoroughly test the system?
17.	Have you noticed whether the customers (users) get any difficulties in understanding the system?	42.	Do you validate customer requirements before releasing the system?
18.	In what way does it show?	43.	In what way and against what?
19.	What is needed to improve the system knowledge for all the roles (developers, support technicians, customers)? 1. Better documentation 2. Other measures in this case what?	44.	Do you have any problems concerning the validation of customer requirements? What are these problems?
20.	Has the lack of or insufficient system documentation lead to more oral communication burden? In what way does it affect the developers' work?	45.	Now, we have tried to identify a set of problems in agile development and find out whether they were relevant in your organization. These problems were strongly related to the documentation practice. Are there any other problems that we might have forgotten? What are they?
21.	Is it possible to estimate the communication burden (in percentage of the overall time)?		
22.	What problems are related to the oral communication of the system and process?		
23.	Does the oral communication lead to the fact that some knowledge may be forgotten or misunderstood?		

the students. Finally, we studied and compiled the results of our interviews to be presented in this paper.

As can be seen in Table 1, our questionnaire was open-ended and semi-structured. The purpose was to give freedom to respondents

to answer in their own terms [20]. Such type of interviewing has a positive effect in a sense that while interviewing one may elicit more knowledge about the studied domain. Its drawback however is the fact that the interviewer must possess good understanding of the domain studied, in order to react to irrelevant answers.

Table 2. List of the organizations studied

	No of employees	Type of products	Size of products	Agile method used	Method used before
Org 1	201	Telephony management	Unknown	ASD	Waterfall
Org 2	200	Games	Unknown	SCRUM	RUP
Org 3	7	Telephony management	Unknown	XP	Waterfall
Org 4	150 000	Telecom Software	Unknown	SCRUM	Increment
Org 5	50	E-business Solutions	Varies	XP	Waterfall
Org 6	11	TV & Cinema Industries	Varies	Various, e.g. XP	None
Org 7	240	Financial Software	500 000 LOC	SCRUM & TDD	Waterfall
Org 8	20	Administrative Systems	Unknown	SCRUM & SP	None
Org 9	600	Multimedia Solutions	10 000-700 000 LOC	TDD, own iterative development, Pair Programming	Waterfall
Org 10	8	Web-bases Systems	2000-8000 LOC	Own agile method	Waterfall
Org 11	3	Process Simulator	10 000 LOC	XP	None
Org 12	30	Web Applications	1-10 Object Points	XP together with Waterfall	Waterfall
Org 13	15	Customized Systems	400 000 LOC	Own agile method	None
Org 14	70	Economy Adds-on	1000-5000 LOC	TDD & Daily Build	Waterfall
Org 15	600	Administrative Systems	3000 Functions	XP influenced RUP	Waterfall
Org 16	10	Web Solutions	Thousands of LOC	Lean method	Waterfall
Org 17	3	DB-bases Systems	Unknown	Crystal Clear	None
Org 18	110	Financial Systems	10 Modules	XP on deadlines	RUP

Due to the fact that we used students in our investigation, we had to be clear about our objectives. To avoid any type of misunderstanding and to allow a comparative study, we created four safeguards against irrelevant answers.

As a first safeguard, we listed and described problems related to documentation in traditional evolution and maintenance. Each problem was thoroughly described, motivated, and explained. As a second safeguard, we made a specification of the expected answers to be provided for each question. We also specified the types of answers that we wished to avoid. Unclear and fuzzy answers would not be accepted and the interviews would have to be repeated. As a third safeguard, each interviewee was asked to provide his name, his role within agile development and contact details to allow the author to contact them with follow-up questions.

Finally, we analyzed the results. We grouped the data collected by the students. To enable the collective analysis of the status for

each of the questions, we created a list of answers to each question. It is this list that we present in this paper.

2.2 Sampling and Validity

The data sampling method was convenience sampling. This means that we did not control the choice of the organizations involved in our study. It was students who did it. Due to the fact that it is difficult to make organizations show willing for an interview, the students were allowed to choose just any organization (large/medium/small and/or private/state) in any country. The only requirement was that the organization studied should use some agile method.

Many of our students, coming from international master program in Sweden, chose organizations in their own countries. Hence, the countries represented in this report are Sweden, Spain, United States, Macedonia, Austria, Portugal, and Romania. These organizations are listed in Table 2. As can be seen there, they practice various forms of agile methods ranging from *eXtreme*

Programming to SCRUM. Due to the sensitivity of the material presented herein, we do not name these organizations.

The small sample size, the type of the organizations studied and the convenience sampling method should not allow us to generalize our results. However, we believe that our paper provides a valuable indication of the problems that have already been encountered in the agile environments.

3. DOCUMENTATION PERSPECTIVE IN TRADITIONAL EVOLUTION AND MAINTENANCE

Documentation, if correct, complete and consistent, has been regarded as a powerful tool for software engineers to gain success. Its purpose is to describe software systems and processes.

Good system documentation thoroughly describes a software system, its components and relationships. It facilitates the understanding and communication of the software system, eases the learning and relearning process, makes software systems more maintainable, and consequently improves the engineer's productivity and work quality [4, 5, 6, 8, 15, 18, 19].

Poor system documentation, on the other hand, is regarded to be the primary reason for quick software system quality degradation and aging [15, 17, 19]. It is a major contributor to the high cost of software evolution and maintenance and to the distaste for software maintenance work [14]. Outdated documentation of a software system misleads the engineer in her work thus leading to confusion in the already very complex evolution and maintenance task.

The purpose of documentation however, is not only to describe a software system but also to record a process. Process documentation is extremely important for achieving (1) insight and visibility into the processes, (2) meaningful process measurement and thereby (3) high process maturity. It should be pervasive in and through all of the types of lifecycle chores. Proper control and management of development, evolution and maintenance can only be achieved if we have obtained sufficient visibility into the process, its stages and tasks, executing roles, their decisions and motivations, and the results of individual process tasks.

Irrespective of whether documentation supports systems or processes, it constitutes a collective knowledge of the organization. It supports communication and collaboration amongst various groups within the organization such as project managers, quality managers, product managers, support technicians, testers, developers, and other roles [1]. It enhances knowledge transfer, preserves historical information, assists ongoing product evolution and maintenance, and fulfills regulatory and legal requirements. It provides an efficient vehicle for communication serving a variety of needs. We distinguish between two types of documentation:

- *Self to self*: Software professionals make personal notes. The goal is to remember what they have done, plan for future actions and analyze the results of their own personal processes.
- *Software professional to software professional*: Different roles communicate with one another, such as developers to

developers, developers to testers, developers to maintainers, developers and/or maintainers to management, management to developers and/or maintainers, developer and/or maintainer to support personnel, and the like. The goal is to provide feedback to one another.

4. PROBLEMS ELICITED WITHIN TRADITIONAL EVOLUTION AND MAINTENANCE

In this section, we describe the problems common in traditional evolution and maintenance environments. We divide them into product and process problems.

4.1 Product Related Problems

System documentation is regarded to be an intrinsic part of any system. Its purpose is not only to communicate on the internals of a software system to be shared among various stakeholders, but also to provide basis for various audits and decisions. In the literature, we have identified the following problems resulting in lack of correct and consistent system documentation. They are (1) developers' system knowledge, (2) sudden loss of critical knowledge in cases of team volatility, (3) training of new hires, (4) support technicians' knowledge, and (5) customer knowledge.

4.1.1 Developers' System Knowledge

Software system documentation is a key component in software quality. Empirical studies have shown that poor or missing documentation is a major cause of defects introduced during evolution and maintenance. Software evolution and maintenance costs tend to be less for well-documented systems than for systems supplied with poor and/or incomplete documentation. [6, 8, 16] Well-documented systems do not namely age in the same tact as the undocumented ones.

Software engineers need to possess good knowledge of the system to be able to continue to evolve it. In an agile environment, there is a risk that most of the documentation is either in the heads of the developers or on temporary posters on the wall. Without proper documentation, there is a high risk that the organizational knowledge of the system can be forgotten, misunderstood or even lost. This in turn may lead to unconsciously inserted faults due to incorrect understanding. Using *Question 11*, we investigate whether the organizations studied have noticed whether their engineers have started losing track and knowledge of the systems they are evolving.

4.1.2 Sudden Loss of Critical Knowledge in Cases of Team Volatility

High personal turnover can also lead to a significant loss of system knowledge if a skilled team member leaves and his knowledge and experience is not recorded. System documentation is regarded to reduce this loss. This is in particular an issue when software needs be evolved and maintained in the long run. Using *Question 13*, we investigate whether and what problems the organizations encounter in cases when key software engineers quit the organization.

4.1.3 Training of New Hires

It is always difficult to train new hires, especially when the only system documentation is code. If there is no system documentation, then the new hires will have to bother her workmates with many questions and thereby distract them from their ordinary work. Using *Question 14*, we investigate how the new hires learn the system, how soon they are allowed to code, and if there are any problems with respect to training of new hires in an agile environment.

4.1.4 Support Technicians' Knowledge

Support technicians need to have good system knowledge for being able to support customers in their daily operation. They may acquire this knowledge either via thorough system documentation, user documentation, or other means. Lack of or inconsistent system documentation and/or user manuals may substantially obstruct their work. Using *Question 15*, we investigate whether the support technicians (on *Support Line 1* and 2) have started losing track and knowledge of the systems they support [11].

4.1.5 Customer System Knowledge

One of the major problems in traditional evolution and maintenance is customers' product knowledge [9, 12]. Customers do not possess the required knowledge of the products they are using. They may use them incorrectly, get incorrect results, lose data, or think that the products they are using are not working correctly. They may report on software problems, which later may show to be operational mistakes. This, in turn, requires extra support effort spent on explaining to the customers that the reported problem was due to their lack of knowledge and experience. Using *Question 17*, we investigate whether the organizations studied have encountered any problems related to the customer system knowledge.

4.2 Process Related Problems

Process documentation enables process monitoring and control. It is essential for many roles involved in it. For instance, managers must have insight into the progress made within the current development process. They must also have insight into the past projects, to be able to make different types of analyses and predictions for future projects.

Using literature, we have identified the following process problems in the traditional evolution and maintenance: (1) inability to monitor and control the process, (2) inability to analyze the process, (3) inability to follow the progress of software engineers, (4) communication complexity, (5) limited ability to make changes, (6) inability to scale up software production, and (7) inability to thoroughly test.

4.2.1 Inability to Monitor and Control the Process

It is regarded that proper control and management of a process can only be achieved if we have obtained sufficient visibility into the development, evolution and maintenance processes, their stages and tasks, executing roles, their decisions and motivations, and the results of each individual process task. Process visibility is claimed to be the main prerequisite for achieving mature processes.

To enable this visibility, processes should be thoroughly documented. Their documentation provides an efficient vehicle for communication serving a variety of needs amongst various groups within the organization such as project managers, quality managers, product managers, support technicians, testers, and other roles [1].

Continuous update of the progress made during the process helps in tracking the process. It simplifies reporting of the progress to various roles involved and helps manage system releases [1].

By documenting the process, the roles involved in it, their workload, effort, and status, we can monitor and control the overall process and effort and estimate the delivery time [1]. This aids in making sure that the plans are followed and that the product will be delivered on time. Using *Questions 24-28*, we investigate whether the organizations document their processes and whether they have sufficient insight into their processes.

4.2.2 Inability to Analyze the Process

By having a documented process, one may easily evaluate the process, identify its shortcomings and make suggestions for its improvement. Documenting of what was implemented or changed, how, why, when, and by who are essential ingredients in audit trails and quality analyses. If we do not document the process, we cannot evaluate it, identify its shortcomings, and make suggestions for its improvement [1]. Using *Question 29-32*, we investigate whether the organizations studied have the ability to analyze their processes.

4.2.3 Inability to Follow the Progress of Individual Software Engineers

Both software engineers and their managers should follow their work progress to be able to assess their effectiveness. Lack of process documentation makes this information invisible, especially to the management. Managers are very reliant on information reporting on the progress of its employees for various reasons. They need it for being able to make plans and various types of estimations, such as cost and effort required for implementing a requirement, and to make sure that the process is being followed by individual engineers. Using *Question 33*, we study whether management may easily follow the progress of each individual engineer.

4.2.4 Communication Complexity

The agilists have traded off explicit knowledge captured in documentation for oral communication and temporary visual control in form of information radiators [2]. Thanks to this, they have achieved more speed within development and evolution. However, the fact that most of the communication is oral may introduce complexity to the communication within the organization and the process management. Also, through oral communication, a lot of knowledge may be forgotten or misunderstood. Using *Questions 20-23*, we investigate (1) whether lack of or insufficient system and process documentation has led to more oral communication, (2) how big is the communication burden, and (3) what problems are encountered in the context of oral communication.

4.2.5 Limited Ability to Make System Changes

Documentation is regarded to be very important when making changes to software systems. One must not only have good knowledge of the system, but also remember the decisions made behind many changes. Hence, maintaining a product that is in use for several years requires both system and process documentation. Using *Question 37*, we investigate whether the organizations studied have encountered problems related to making system changes.

4.2.6 Inability to Scale up Software Production

Agile methods may be effective in developing, evolving and maintaining software in small organizations. However, as products become larger and more complex and small organizations expand, their software production scales up as well. The organizations may scale up from having one team to several teams. Communication among those teams may become awkward, and communication is needed for various purposes such as, for instance, to agree upon the interfaces. Using *Questions 38-40*, we investigate whether the organizations studied have encountered any problems when scaling up their software systems, organizations, and production, and how they have tried to remedy these problems.

4.2.7 Inability to Thoroughly Test

In traditional evolution and maintenance, it is a known problem that testers do not manage to thoroughly test the system. The reason is often time and effort required for achieving sufficient system testing coverage. We suspect that this problem stays in agile environment as well, especially when several teams are involved and a separate testing team is created. Due to short releases, the testers may not have any opportunity to properly prepare the test plans and to thoroughly test the software. Also, lack of system documentation may also mean that they have to base their testing work on oral communication. Using *Question 41*, we investigate whether the organizations have noticed any problems with respect to their ability to test their systems.

5. INTERVIEW RESULTS

In this section, we describe the results of our interviews. We first provide a general description of the organizations studied in Section 5.1. We then present the status in Sections 5.2 and 5.3, respectively.

5.1 Presentation of Organizations Studied

We have studied 18 software organizations practicing some form of an agile method. As illustrated in Table 2, twelve organizations are small enterprises. The remaining ones are big. The systems they produce vary from games, to various administrative, telephony, business and web-based systems. Their size ranges from 1000 – 700 000 LOC. The agile methods used are SCRUM, Lean, XP, TDD, Crystal Clear, ASD, and various home-made agile methods. The roles involved in our interviews were developers, team leaders, project managers, client representatives, IT managers, senior consultants, chief architects, and executive managers.

Most of the organizations studied have some experience of traditional evolution and maintenance processes. The models used by them before were mainly RUP and various kinds of Waterfall-

based models. Five out of 18 organizations studied did not have experience of heavyweight plan-driven processes.

5.2 Product Related Problems

5.2.1 System Knowledge

Fifty percent (nine out of 18) of the organizations studied claim that their software engineers start losing track and knowledge of the system(s) they are evolving. It shows in different ways: (1) code becomes complex, (2) architecture deteriorates, (3) engineers are not certain of what changes to make and where, (4) they ask questions revealing lack of knowledge, and (5) they repeat many of the former mistakes.

Two of these organizations claim that the deteriorated architecture was the most serious problem. Why it deteriorates is the fact that engineers make changes that do not fit the “*grand architectural vision*”. Hence, it is important to find a means to effectively share the system architecture. These two organizations claim that one way of achieving it is to have good documentation of system architecture. In addition, the other of these two organizations, has assigned an architect working 50% on assuring the architectural quality.

Regarding the remaining organizations (the remaining 50%), they have not noticed that their engineers have started losing system knowledge. The reasons to this are the following: (1) the companies are too young, they have only existed for a few years, (2) the companies are very small; they have only 1-3 engineers who have developed, evolved and maintained the system from the very beginning, or (3) the companies continuously update system knowledge via Scrum meetings.

5.2.2 Sudden Loss of Critical Knowledge in Cases of Team Volatility

Most of the organizations studied claim that they are too young. Hence, they have not managed to experience the team volatility problem. However, twelve out of 18 organizations are of the opinion that a sudden loss of key engineers would be a serious problem. It would result in (1) lack of substantial system knowledge, (2) initial pain of making changes to the system, (3) big challenge to find replacements, (4) productivity decrease, and (4) increased production cost.

The remaining organizations have guarded themselves against this problem. They use different safeguards. Three organizations follow the principle of common system ownership. By frequently rotating engineers, the organizations have become less vulnerable to this problem. Two other organizations set aside time for knowledge transfer, either in an oral or written form (documentation).

5.2.3 Training New Hires

In most of the organizations studied, the newcomers are allowed to code soon after they are hired. The time span varies, from immediately on the very first day to more than three months. This is of course strongly related to the complexity of the systems maintained. In three organizations, the motto followed is “*start practicing instead of noticing*”. Here, the new hires start coding immediately. They pair-program together with their mentors.

In twelve organizations, the new employees are allowed to code between two to four weeks after they got hired. They learn the system by (1) reviewing the code, (2) reading the documentation in the companies having the documentation, (3) testing, (4) intensive oral communication, and (5) mentorship via pair programming. In the remaining three organizations, it may take at least three and at most one year to fully use newcomers in programming activities. This is due to the complexity of their systems.

5.2.4 *Support Technicians' Knowledge*

Only in three out of 18 organizations studied, support technicians have experienced difficulties when supporting their customers on a daily basis. They do not always manage to learn the system they support. In six other organizations, it is the developers who conduct support. Hence, this problem is not relevant for them. Two other organizations, having separate support, do not experience this problem at all. Finally, the remaining seven organizations have not provided us with clear answers. Hence, we cannot make any conclusions regarding their support status.

5.2.5 *Customer System Knowledge*

Six out of 18 organizations experience some problems related to their customers' system knowledge. The reason is lack of sufficiently documented user manuals. Many times, the problem concerns the knowledge of secondary functionalities. The organizations try to resolve this problem via additional support or various demonstrations.

The remaining organizations have not experienced this problem at all. The reasons are (1) continuous contact with the customer, (2) user-friendly self-documenting systems.

5.3 **Process Related Problems**

5.3.1 *Inability to Monitor and Control the Process*

Six out of 18 organizations do not document their evolution and maintenance processes at all. These are mainly small organizations. Three of these six organizations claim that they have good insight into the process via frequent meetings.

Four out of 18 organizations have partial visibility into the process. This is because they do not fully document their processes. The process steps that mainly get documented concern release management, and project management. Finally, the remaining eight organizations document most of the steps within their processes. Hence, they claim that they are able to monitor and control them.

5.3.2 *Inability to Analyze the Process*

Five out of 18 organizations studied did not provide us with any answer whether they may analyze their evolution and maintenance processes. One of the remaining 13 organizations does not analyze their processes at all. Regarding the remaining twelve organizations, they do process analysis on a periodic basis, either via retrospective meetings, or some other form of weekly or monthly meetings.

5.3.3 *Inability to Follow the Progress of Individual Software Engineers*

Four out of 18 organizations studied cannot follow the progress of each individual engineer. In the 14 remaining organizations, one may do it partially. Out of these 14 organizations, three of them follow the individual engineer progress via meetings. In the remaining eleven organizations, one does it via a tool supporting the organizational process.

5.3.4 *Communication Complexity*

Fifty percent of the organizations studied claim that insufficient documentation has lead to an oral communication burden. The communication takes time, often stealing it from other work assignments. To add zest to it, it may even be time consuming to find a person having the right answers. Regarding the remaining organizations, they claim that communication overhead has increased. However, they do treat it as part of their job. In two organizations, they have even defined roles responsible for communicating, the so-called *Communication Owners*.

Seven of the organizations studied have difficulties in estimating the communication burden. Six other organizations spend about 10-20% of total time on communicating. Regarding the remaining six organizations studied, the communication burden ranges from 25% to 50%.

Thirteen out of 18 organizations studied claim that they have encountered various problems directly related to oral communication. These problems are: (1) engineers forget as time passes; they do not remember how they have solved various problems, (2) lack of system understanding, (3) undocumented decisions and process activities create problems, (4) many product or process issues get misunderstood, (5) people do not remember what they said, what they were told, or asked for, (6) difficulties to find a person knowing the answer, (7) engineers get too much distracted when having to communicate.

5.3.5 *Limited Ability to Make System Changes*

Thirteen out of 18 organizations studied encounter problems when making changes to software systems. They claim that it has become more difficult to evolve and maintain their systems. The reasons are (1) it is difficult to change code that was originally developed by someone else, (2) it is difficult to remember which changes have already been made, and (3) knowledge of the system or its parts required for making a change is spread in the heads of various engineers. Regarding the remaining five organizations, they are either too small or too young to have experienced this as a problem.

5.3.6 *Inability to Scale up Software Production*

Seven out of 18 organizations studied have grown with respect to their product size and organization. The remaining organizations have not grown much. Hence, this issue is not applicable for them.

Regarding the first seven organizations, they have noticed that (1) the system has become more difficult to understand and change, (2) teams have become more difficult to coordinate, (3) engineers encountered communication problems when there were more than ten people involved, and (4) engineers were losing track of the company's vision.

For four of these seven organizations, the growth was not a problem. They found that many agile practices were scalable. They have remedied the scalability problems by strengthening the process with (1) system ownership by splitting the teams and assigning them clear ownership of system parts, (2) evolution and maintenance coordinators dedicated to coordinating work in teams and with other teams, (3) follow-up meetings, (4) better process documentation, and (5) better system documentation.

5.3.7 Inability to Thoroughly Test

In twelve out of 18 organizations studied, testers do not encounter any problems related to testing the system in agile environment. In four of these twelve organizations, developers play the role of testers as well. In the remaining eight of these twelve organizations, one does not experience this problem due to the following reasons: (1) one has good systems documentation including use cases (2) testers are integrated with developers, and (3) the organizations use support of daily build process.

Six organizations experience problems with testing. They state that testing takes too long time. Due to the fact that information is orally conveyed, testers often misunderstand the functionality to be tested.

6. FINAL REMARKS

In this paper, we have studied 18 software organizations practicing agile methods. Using a list of well-known problems in traditional evolution and maintenance, we investigated whether similar problems have been encountered in agile trenches.

Although our study covers a substantial amount of organizations, it does not allow us to generalize our results. This is because we used a convenience sampling method, and most of the organizations studied and their systems are still young. Hence, it is too early to identify the lifecycle impact of agile methods in these organizations. However, we believe that our results provide an indication of problems that may be encountered in evolution and maintenance trenches. These problems are:

- Oral communication takes time.
- Oral communication distracts too much and steals valuable time.
- It is difficult to find a person who has the right answer.
- Oral communication is the reason why engineers forget the system and the decisions made. Engineers do not remember what they said, what they were told, or asked for.
- Engineers are not certain of what changes to make and where.
- Engineers do not remember what changes they made and why.
- Engineers ask questions revealing lack of system knowledge.
- Engineers repeat many of the former mistakes.
- Organizations may suddenly lose valuable knowledge in cases when key engineers quit.
- Organizations may undergo major initial pain in their change process after key engineers quit. The pain is defined in terms of loss of time and profit.
- Engineers do not know the whole system. Knowledge of the system is spread in the heads of several engineers.
- Architecture deteriorates substantially.
- Many product and process issues get misunderstood.

- When scaling up, teams have become more difficult to coordinate, systems have become more difficult to understand and change, and engineers have started losing track of the company's vision.
- Testers do not manage to thoroughly test the system.
- Due to oral communication, testers misunderstand the functionality to be tested.

Other problems as recognized by the organizations studied were (1) challenge to move from plan-driven method to the agile one, (2) unit testing does not get enough attention.

When asking the organizations for advice on how to improve system and process knowledge, twelve out of 18 organizations studied stated that one should strive to have better system and process documentation. Some of the interviewees claimed that the main problem with agile processes was the fact that engineers did not pay enough attention to documentation. According to the majority of our interviewees, documentation is a prerequisite for sustainable organization.

Some of the organizations studied have undertaken measures to remedy some of the above-mentioned problems. These measures are:

- To remedy the architectural ad hoc expansion, the organizations have created an *Architect* role assuring the architectural quality. This measure is nothing new and is commonly practiced in traditional environments.
- To remedy the communication problem, the organizations have created a *Communication Owner* role, a role responsible for communicating the overall system and process information. The existence of this role however does not guarantee that the valuable information gets documented. This measure is new with respect to traditional environment.
- To remedy the scalability problems, the organizations have utilized the practices from traditional evolution and maintenance by creating system/component ownership role. They have also improved their system and process documentation.

7. EPILOGUE

Even if it is still too early to draw conclusions on the lifecycle impact of agile methods, this study has provided us with some indications of the problems that may be encountered in agile environments. More studies however need to be made not only to confirm the results presented herein but also to find some kind of a compromise that facilitates development, evolution and maintenance, and that does not burden the engineers with the recording effort. Maybe, an introduction of a *Communication Owner* would be a good compromise. His task would be to communicate the system and the process both orally and in written.

Introducing the role of a *Communication Owner* however does not solve the problem of determining the extent and scope of required documentation. Still, the software community is confused on issues such as (1) what should we document, (2) how much, (3) when, and (4) for what purpose? Today, we do not have any concrete answers. We only claim that we should document just enough information for supporting the communication. But, how much enough is enough?

So far, very little research has been done on documentation. To our knowledge, there are only a few publications reporting on this subject [10, 21-26]. Hence, we claim that this domain is strongly under-explored. We believe that substantial research is required for agreeing upon the scope and extent of documentation so that both short-term and long-term benefits may be gained.

8. REFERENCES

- [1] Arthur, L. J. 1995. *Software Evolution: The Software Maintenance Challenge*. John Wiley & Sons.
- [2] Beck K., 2004. *Extreme Programming Explained: Embrace Change*. 2nd Edition. Upper Saddle River, NJ, Addison-Wesley.
- [3] Cohn M., 2006. *Agile Estimating and Planning*. Pearson Education, Upper Saddle River, NJ.
- [4] Bauer, B. J., and Parnas, D. L. 1995. Applying mathematical software documentation: An experience report, proceedings. Proc. 10th Annual Conference on Computer Assurance. 273–284.
- [5] Briand, L. C. 2003. Software documentation: How much is enough? Proc. IEEE Int. Conf. Softw. Maint. Reeng. 13–15.
- [6] Card, D., McGarry, F., and Page, G. 1987. Evaluating software engineering technologies. IEEE Trans. Softw. Eng. 13(7): 845–851.
- [7] Chapin, N. 1985. Software maintenance: A different view. AFIPS Proc. Natl. Comput. Conf. AFIPS Press, Reston, Virginia, 54, pp. 508 – 513.
- [8] Conwell, C. L. 2000. Capability maturity models support of modeling and simulation verification, validation, and accreditation. Proceedings, IEEE Winter Simulation Conference. 819–828.
- [9] Holt, P. O. 1993. System documentation and system design: A good reason for designing the manual first. Proc. IEEE Colloquium on Issues in Computer Support for Documentation and Manuals. 1/1 – 1/3.
- [10] Kajko-Mattsson, M. 2005. A Survey of Documentation Practice within Corrective Maintenance, Empirical Software Engineering Journal, Kluwer, Volume 10, Issue 1, January, pp. 31 – 55.
- [11] Kajko-Mattsson, M. 2007. Maturity Status within Front-End Support Organisations, International Conference on Software Engineering, IEEE, Computer Society Press: Los Alamitos, CA.
- [12] Kantner, L., et al. 2002. Structured heuristic evaluation of online documentation. Proc. IEEE Int. Prof. Commun. Conf. 331–342.
- [13] Lientz, B. P., and Swanson, E. B. 1980. *Software Maintenance Management, A study of the Maintenance of Computer Application Software in 487 Data Processing Organizations*. Addison-Wesley.
- [14] Martin, J., and McClure, C. 1983. *Software Maintenance, The Problem and Its Solutions*. Englewood Cliffs, New Jersey: Prentice-Hall, Inc. 07632.
- [15] Parnas, D. L. 2000. Requirements documentation: Why a formal basis is essential. Proc. IEEE 4th International Conference Requirements Engineering, pp. 81–82.
- [16] Pigoski, T. M. 1997. *Practical Software Maintenance*. John Wiley & Sons.
- [17] Sousa, M., and Mendes Moreira, H. 1998. A survey of the software maintenance process. In Proceedings, IEEE International Conference on Software Maintenance, pp. 265–272.
- [18] Visaggio, G. 2001. Aging of a data-intensive legacy system: Symptoms and remedies. Journal of Software Maintenance and Evolution: Research and Practice. Wiley & Sons, 13(5): 281 – 308.
- [19] Visconti, M., and Cook, C. R. 2002. An overview of industrial software documentation practice. Proc. 12th IEEE International Conference of the Chilean Computer Science Society, pp. 179–186.
- [20] Walker, R. (ed.) 1985. *Applied Qualitative Research*. Gower Publishing Company Ltd.
- [21] Clark PG., Lobsitz RM, Shields JD., 1989. Documenting the Evolution of an Information System, In Proceedings, IEEE National Aerospace and Electronic Conference, pp. 1819-1826.
- [22] Lepasaar M, Varkoi T, Jaakkola H., 2001. Documentation as a Software Process Capability Indicator, In Proceedings, International Conference on Management of Engineering and Technology, IEEE, Vol. 1, p. 436.
- [23] Norman RL, Holloran RW. 1991. How to simplify the structure of administrative procedures to make them easier to write, review, produce, and use, In Proceedings, International Conference on Professional Communication, Vol.2 pp. 447 – 450.
- [24] Parnas D L. 2000. Requirements documentation: why a formal basis is essential, In Proceedings, 4th International Conference on Requirements Engineering, IEEE, pp. 81-82.
- [25] van Schouwen AJ, Parnas DL, Madey J. 1993. Documentation of Requirements for Computer Systems, In Proceedings, IEEE International Symposium on Requirements Engineering, pp. 198-207.
- [26] Visconti, M., Cook, CR. 2000. An Overview of Industrial Software Documentation Practices, Technical Report 00-60-06, Computer Science Department, Oregon State University.